Experiment No. – 2				
Date of Performance:	**22/1/2026**			
Date of Submission:	**29/1/2026**			
Execution (06)	Viva (02)	Professional Ethics (02)	Experiment Total (10)	Sign with Date

Experiment No. 2

2.1 Aim: Write simple program using solidity language and deploy on truffle local blockchain

2.2 Course Outcome: Develop the application using the concept of Ethereum with smart contracts using different development tools.

2.3 Learning Objectives: Deploy solidity language program on truffle local blockchain.

2.4 Requirement:

- Computer / Laptop
- Operating System: Windows / Linux / macOS
- Node.js and npm
- Truffle Suite
- Ganache
- Solidity programming language
- Code editor (Visual Studio Code)
- Web browser

2.5 Related Theory:

Introduction to Solidity

Solidity is a contract-oriented, high-level programming language used for writing smart contracts. It is specifically designed to run on the Ethereum Virtual Machine (EVM) and is one of the most widely used languages for blockchain-based application development.

Solidity syntax is influenced by popular programming languages such as C++, Python, and JavaScript, making it easier for developers to learn and use. Smart contracts written in Solidity define the rules and logic of decentralized applications (DApps) and are stored permanently on the blockchain.

What is a Smart Contract?
A smart contract is a self-executing program that runs on a blockchain network. The contract automatically executes predefined actions when specified conditions are met, without the need for intermediaries.
In Solidity, a smart contract is similar to a class in Java. It contains:

- State variables (data)
- Functions (methods)
- Events and modifiers

Once deployed, the contract resides on the blockchain and cannot be altered.

What is a Contract in Solidity?
A contract in Solidity is a collection of:

- Variables that store the state of the contract
- Functions that define the behavior of the contract

Contracts are stored on the Ethereum blockchain and are executed by the Ethereum Virtual Machine. Each contract has its own address and can interact with other contracts and external accounts.

Solidity Source Code File Extension
Solidity source code files use the .sol extension.
Example:
HelloWorld.sol – a simple Solidity smart contract file.
Solidity contracts can be written and edited using any text editor or IDE such as VS Code, Remix IDE, or Atom.

Advantages of Solidity

- Object-oriented language supporting contracts, inheritance, and libraries
- High-level and human-readable syntax
- Strong and growing developer community
- Availability of multiple development tools (Truffle, Remix, Hardhat)
- Statically typed language, ensuring type safety
- Designed specifically for smart contract development
- Executes on the Ethereum Virtual Machine (EVM)

Disadvantages of Solidity

- Smart contracts are immutable after deployment; modifying existing contracts is not possible
- Solidity is relatively new, so bugs and security risks may exist
- Does not support floating-point data types
- Errors in code can lead to permanent financial loss

Solidity Compilation Process
The Solidity compilation process involves converting human-readable source code into machine-readable bytecode that can be executed on the blockchain.
The steps involved are:

1. Writing the Contract
   The smart contract is written in a .sol file using Solidity syntax.

2. Compilation
   The Solidity compiler (solc) compiles the source code into:
   - Bytecode – executed by the Ethereum Virtual Machine
   - ABI (Application Binary Interface) – defines how external applications interact with the contract
3. Deployment
   Using Truffle, the compiled contract is deployed to a blockchain network (local or test network).
4. Execution
   Once deployed, contract functions can be called, and transactions are executed on the blockchain.

Role of Truffle in Deployment
Truffle is a development framework that simplifies:
- Contract compilation
- Deployment using migration scripts
- Network configuration
- Testing smart contracts

Using Truffle ensures structured development and reduces errors during deployment.

2.6 Procedure:

- Install Node.js and npm on the system.
- Install Truffle Suite using npm.
- Install and launch Ganache to start a local blockchain.
- Create a new Truffle project using the command truffle init.
- Write a simple Solidity smart contract and save it in the contracts folder.
- Create a migration file in the migrations folder for deploying the contract.
- Configure the Truffle network settings to connect with Ganache.
- Compile the smart contract using truffle compile.
- Deploy the contract on the local blockchain using truffle migrate.
- Verify successful deployment by checking the contract address and transaction details.

2.7 Program and Output:

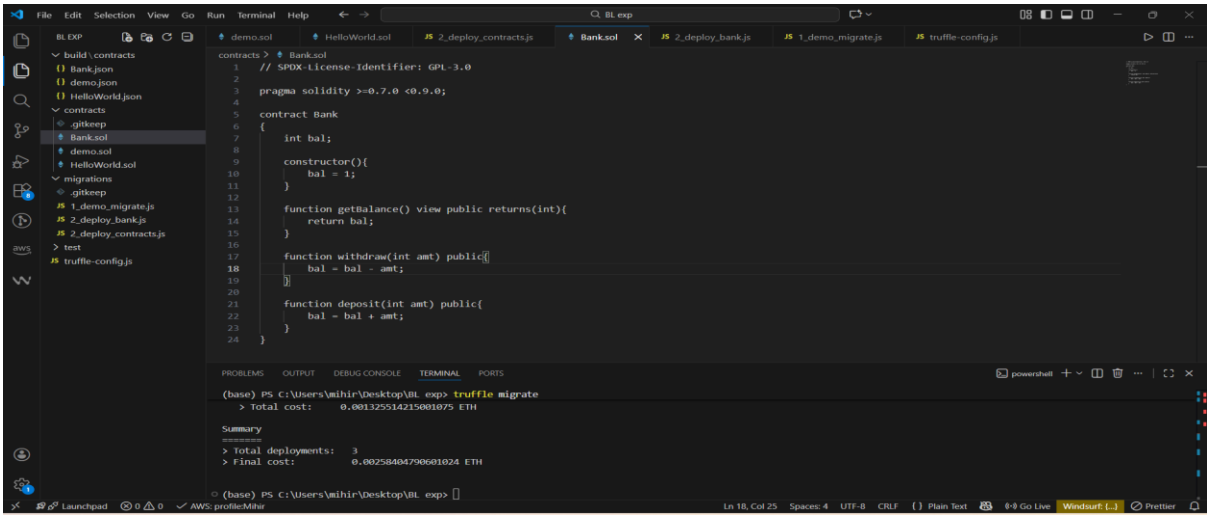
Bank.sol:

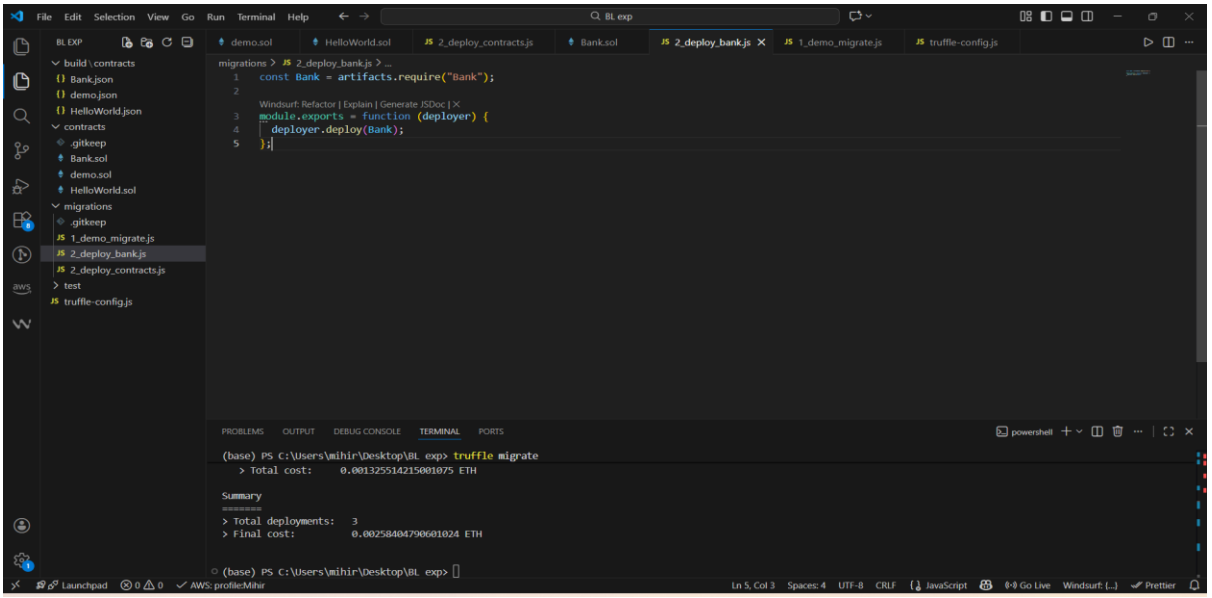
```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Bank
```

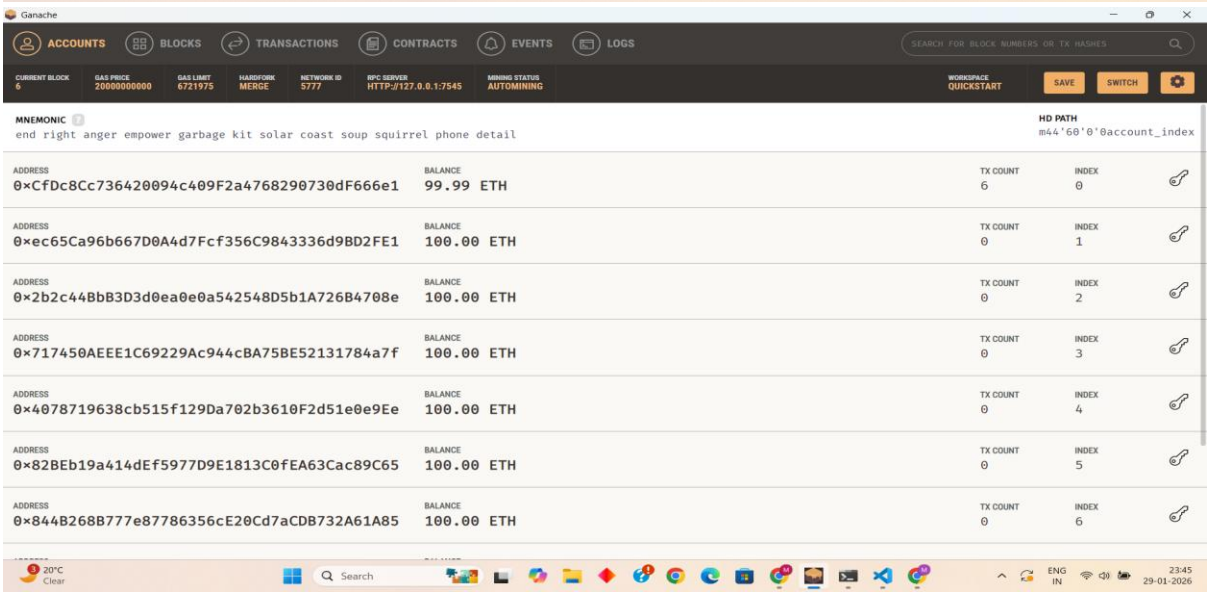
```
{
   int bal;

   constructor(){
      bal = 1;
   }

   function getBalance() view public returns(int){
      return bal;
   }

   function withdraw(int amt) public{
      bal = bal - amt;
   }

   function deposit(int amt) public{
      bal = bal + amt;
   }
}
```

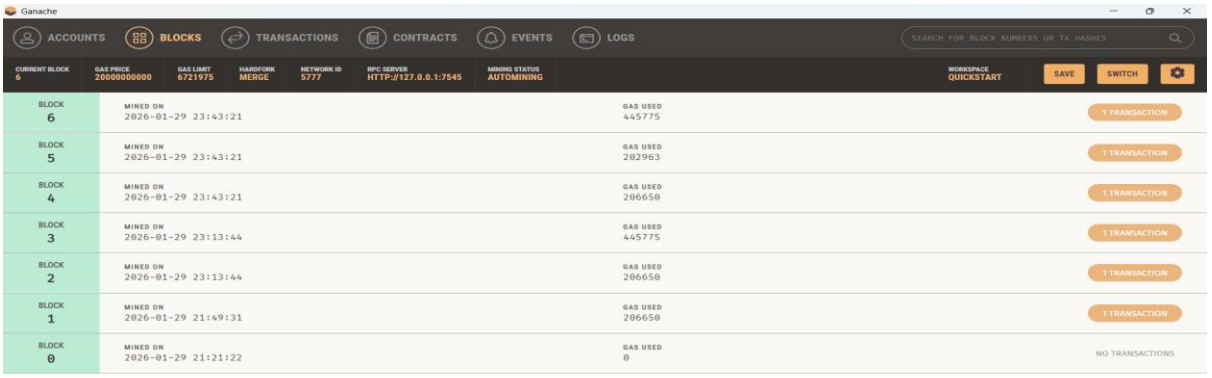
_deploy_bank.js:

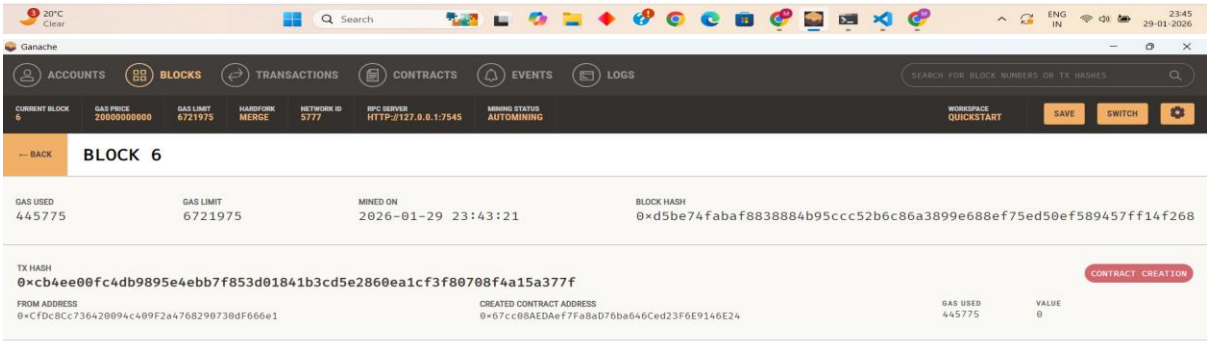
```
const Bank = artifacts.require("Bank");


module.exports = function (deployer) {
  deployer.deploy(Bank);
};
```

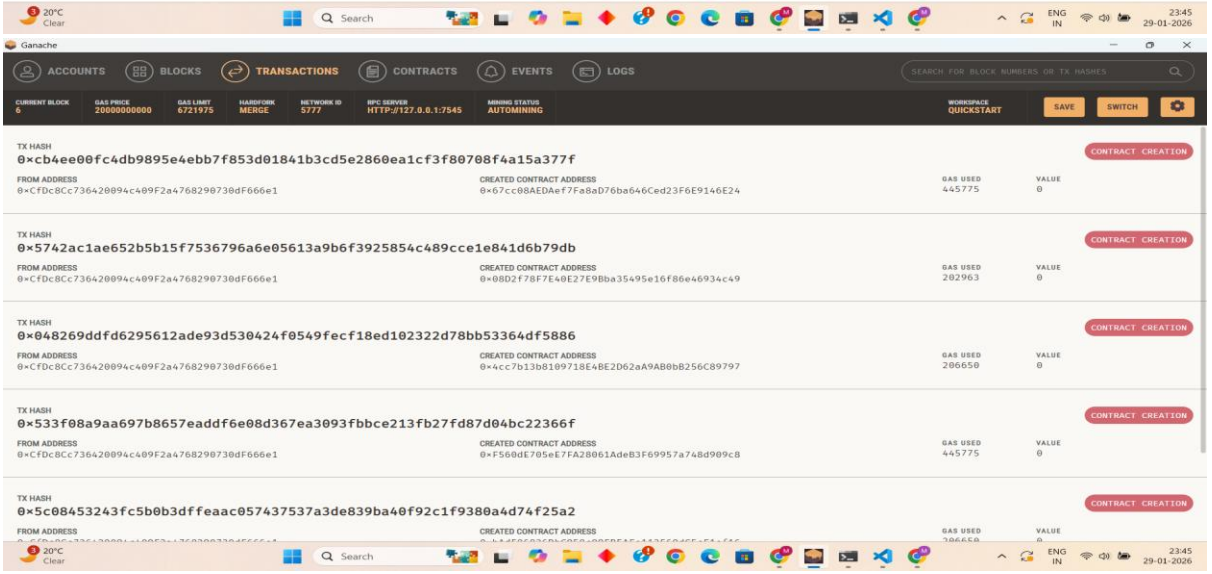

// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract Bank
{
int bal;
constructor(){
bal = 1;
}
function getBalance() view public returns(int){
return bal;
}
function withdraw(int amt) public{
bal = bal - amt;
}
function deposit(int amt) public{
bal = bal + amt;
}
}
(base) PS C:\Users\mihir\Desktop\BL exp> truffle migrate
> Total cost: 0.00132551421500107 5 ETH
Summary
> Total deployments: 3
> Final cost: 0.0025840479060102 4 ETH

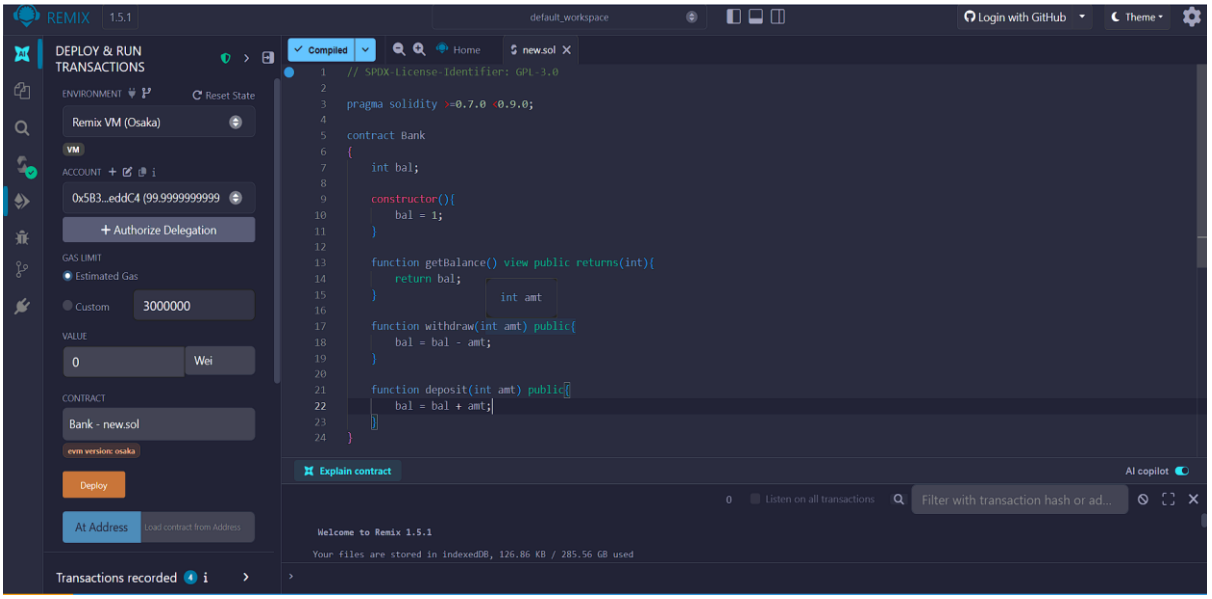

const Bank = artifacts.require("Bank");
module.exports = function (deployer) {
deployer.deploy(Bank);
};
(base) PS C:\Users\mihir\Desktop\BL exp> truffle migrate
> Total cost: 0.00132551421500107 5 ETH
Summary
> Total deployments: 3
> Final cost: 0.0025840479060102 4 ETH

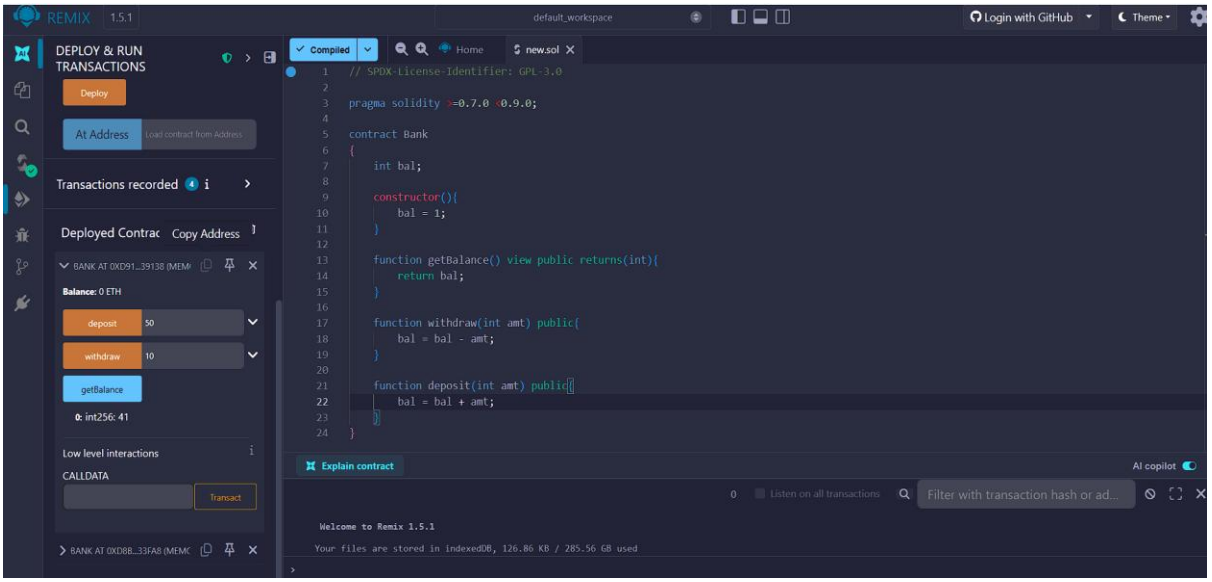

ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
MNEMONIC
end right anger empower garbage kit solar coast soup squirrel phone detail
HD PATH
m/44'/60'/0'/0/account_index
0xCfDc8Cc736420094c409F2a4768290730dF666e1 99.99 ETH
0xec65Ca96b667D0A4d7Fcf356C9843336d9BD2FE1 100.00 ETH
0x2b2c44BbB3D3d0ea0e0a542548D5b1A726B4708e 100.00 ETH
0x717450AEEE1C69229Ac944cBA75BE52131784a7f 100.00 ETH
0x4078719638cb515f129Da702b3610F2d51e0e9Ee 100.00 ETH
0x82BEb19a414dEf5977D9E1813C0fEA63Cac89C65 100.00 ETH
0x844B268B777e87786356cE20Cd7aCDB732A61A85 100.00 ETH

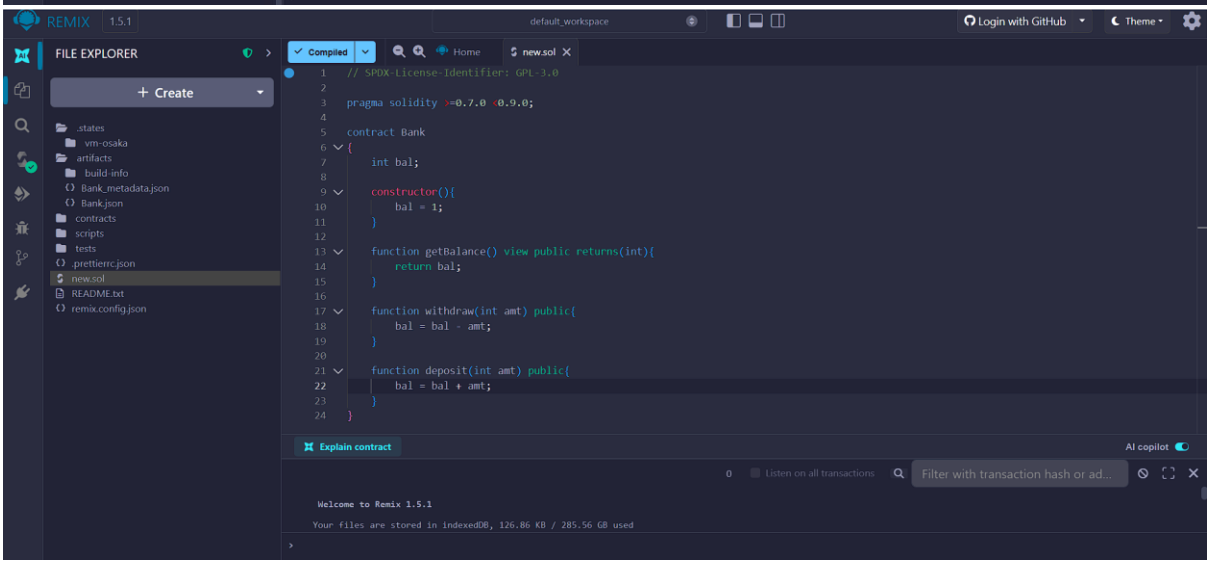

Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 6
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
BLOCK 6 MINED ON 2026-01-29 23:43:21 GAS USED 445775 1 TRANSACTION
BLOCK 5 MINED ON 2026-01-29 23:43:21 GAS USED 202963 1 TRANSACTION
BLOCK 4 MINED ON 2026-01-29 23:43:21 GAS USED 206650 1 TRANSACTION
BLOCK 3 MINED ON 2026-01-29 23:13:44 GAS USED 445775 1 TRANSACTION
BLOCK 2 MINED ON 2026-01-29 23:13:44 GAS USED 206650 1 TRANSACTION
BLOCK 1 MINED ON 2026-01-29 21:49:31 GAS USED 206650 1 TRANSACTION
BLOCK 0 MINED ON 2026-01-29 21:21:22 GAS USED 0 NO TRANSACTIONS


Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 6
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
← BACK
BLOCK 6
GAS USED 445775
GAS LIMIT 6721975
MINED ON 2026-01-29 23:43:21
BLOCK HASH 0×d5be74fabaf8838884b95ccc52b6c86a3899e688ef75ed50ef589457ff14f268
TX HASH 0×cb4ee00fc4db9895e4ebb7f853d01841b3cd5e2860ea1cf3f80708f4a15a377f
CONTRACT CREATION
FROM ADDRESS 0×CfDc8Cc736420094c409F2a4768290730dF666e1
CREATED CONTRACT ADDRESS 0×67cc08AEDAef7Fa8aD76ba646Ced23F6E9146E24
GAS USED 445775
VALUE 0


Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 6
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
TX HASH 0×cb4ee00fc4db9895e4ebb7f853d01841b3cd5e2860ea1cf3f80708f4a15a377f
CONTRACT CREATION
FROM ADDRESS 0×CfDc8Cc736420094c409F2a4768290730dF666e1
CREATED CONTRACT ADDRESS 0×67cc08AEDAef7Fa8aD76ba646Ced23F6E9146E24
GAS USED 445775
VALUE 0
TX HASH 0×5742ac1ae652b5b15f7536796a6e05613a9b6f3925854c489cce1e841d6b79db
CONTRACT CREATION
FROM ADDRESS 0×CfDc8Cc736420094c409F2a4768290730dF666e1
CREATED CONTRACT ADDRESS 0×08D2f78F7E40E27E9Bba35495e16f86e46934c49
GAS USED 202963
VALUE 0
TX HASH 0×048269ddfd6295612ade93d530424f0549fecf18ed102322d78bb53364df5886
CONTRACT CREATION
FROM ADDRESS 0×CfDc8Cc736420094c409F2a4768290730dF666e1
CREATED CONTRACT ADDRESS 0×4cc7b13b8109718E4BE2D62aA9AB0bB256C89797
GAS USED 206650
VALUE 0
TX HASH 0×533f08a9aa697b8657eaddf6e08d367ea3093fbbce213fb27fd87d04bc22366f
CONTRACT CREATION
FROM ADDRESS 0×CfDc8Cc736420094c409F2a4768290730dF666e1
CREATED CONTRACT ADDRESS 0×F560dE705eE7FA28061AdeB3F69957a748d909c8
GAS USED 445775
VALUE 0
TX HASH 0×5c08453243fc5b0b3dffeaac057437537a3de839ba40f92c1f9380a4d74f25a2
CONTRACT CREATION
FROM ADDRESS
CREATED CONTRACT ADDRESS
GAS USED
VALUE


REMIX 1.5.1
DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Reset State
Remix VM (Osaka)
VM
ACCOUNT
0x5B3...eddC4 (99.9999999999
+ Authorize Delegation
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
Bank - new.sol
evm version: osaka
Deploy
At Address
Load contract from Address
Transactions recorded
Compiled
Home
new.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract Bank
{
int bal;
constructor(){
bal = 1;
}
function getBalance() view public returns(int){
return bal;
}
function withdraw(int amt) public{
bal = bal - amt;
}
function deposit(int amt) public{
bal = bal + amt;
}
}
Explain contract
AI copilot
Welcome to Remix 1.5.1
Your files are stored in indexedDB, 126.86 KB / 285.56 GB used


REMIX 1.5.1
DEPLOY & RUN TRANSACTIONS
Deploy
At Address
Transactions recorded
Deployed Contrac
Copy Address
BANK AT 0XD91...39138 (MEMO
Balance: 0 ETH
deposit
50
withdraw
10
getBalance
0: int256: 41
Low level interactions
CALLDATA
Transact
BANK AT 0XD8B...33FA8 (MEMO
Compiled
Home
new.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract Bank
{
int bal;
constructor(){
bal = 1;
}
function getBalance() view public returns(int){
return bal;
}
function withdraw(int amt) public{
bal = bal - amt;
}
function deposit(int amt) public{
bal = bal + amt;
}
}
Explain contract
AI copilot
Welcome to Remix 1.5.1
Your files are stored in indexedDB, 126.86 KB / 285.56 GB used


REMIX 1.5.1
FILE EXPLORER
+ Create
.states
vm-osaka
artifacts
build-info
Bank_metadata.json
Bank.json
contracts
scripts
tests
.prettierrc.json
new.sol
README.txt
remix.config.json
Compiled
Home
new.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract Bank
{
int bal;
constructor(){
bal = 1;
}
function getBalance() view public returns(int){
return bal;
}
function withdraw(int amt) public{
bal = bal - amt;
}
function deposit(int amt) public{
bal = bal + amt;
}
}
Explain contract
AI copilot
Welcome to Remix 1.5.1
Your files are stored in indexedDB, 126.86 KB / 285.56 GB used

2.8 Conclusion:

Solidity is a powerful language for developing secure and decentralized smart contracts on the Ethereum platform. Writing and deploying a simple Solidity program using Truffle helps in understanding the fundamentals of smart contract development, compilation, and deployment on a blockchain network.